



南開大學
Nankai University

计算机学院
深度学习实验报告

循环神经网络实验报告

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 6 月 14 日

目录

1	实验要求	2
2	实验环境与数据处理	2
3	原始版本 RNN	3
3.1	网络结构	3
3.2	训练流程	3
3.3	结果与分析	4
4	LSTM	5
4.1	网络结构	5
4.2	训练结果	6
5	LSTM 优势分析	7
6	GRU	8
6.1	网络结构	8
6.2	训练结果	9
7	对比分析	10
7.1	结构对比	10
7.2	指标对比	11
7.3	预测样例	12
8	实验心得	13

1 实验要求

- 掌握 RNN 原理；
- 学会使用 PyTorch 搭建循环神经网络来训练名字识别；
- 学会使用 PyTorch 搭建 LSTM 网络来训练名字识别；
- 个人实现 LSTM 和 GRU 网络结构，并在相同验证集上比较 RNN、LSTM 和 GRU 的实验结果。

2 实验环境与数据处理

本实验基于教师提供的 `rnn_pytorch_tutorial_ori.ipynb` 完成。数据集为 `names` 目录下的 18 个语种姓名文本文件，共包含 16069 个训练样本和 4005 个验证样本。每个字符被编码为长度为 57 的 one-hot 向量，姓名被表示为形状为 `line_length × 1 × 57` 的张量。我使用 `cuda` 设备训练。

为了满足“在名字识别验证集上训练并评估”的要求，我对原始教程代码做了如下调整：

1. 将每个语种的姓名按 8:2 划分为训练集和验证集，保证 RNN、LSTM 和 GRU 使用同一验证集；
2. 原始 notebook 中 LSTM 会覆盖变量 `rnn`，因此将 RNN、LSTM、GRU 分别保存为不同模型变量；
3. 在训练过程中每 1000 次迭代记录训练 loss、验证 loss 和验证准确率，每 5000 次迭代打印一次当前结果；
4. 混淆矩阵统一在验证集 `val_examples` 上统计；
5. LSTM 和 GRU 均采用手写门控结构，没有直接调用 `nn.LSTM` 或 `nn.GRU`。

核心数据划分代码如下。

```
1 def split_by_category(category_lines, validation_ratio=0.2):
2     train_lines = {}
3     val_lines = {}
4
5     for category, lines in category_lines.items():
6         shuffled = lines[:]
7         random.shuffle(shuffled)
8         n_val = max(1, int(len(shuffled) * validation_ratio))
9         val_lines[category] = shuffled[:n_val]
10        train_lines[category] = shuffled[n_val:]
11
12    return train_lines, val_lines
13
14 train_category_lines, val_category_lines = split_by_category(category_lines)
15 val_examples = [
16     (category, line)
17     for category in all_categories
18     for line in val_category_lines[category]
19 ]
20
```

数据读取与划分单元格输出如下。

```
Using device: cuda
n_categories = 18
train examples = 16069
validation examples = 4005
```

3 原始版本 RNN

3.1 网络结构

原始 RNN 使用当前字符输入 x_t 和上一时刻隐藏状态 h_{t-1} 拼接后的向量作为输入，分别经过两个线性层得到新的隐藏状态和类别输出。最后通过 LogSoftmax 输出 18 个语种的对数概率。

```
1 class RNN(nn.Module):
2     def __init__(self, input_size, hidden_size, output_size):
3         super(RNN, self).__init__()
4         self.hidden_size = hidden_size
5         self.i2h = nn.Linear(input_size + hidden_size, hidden_size)
6         self.i2o = nn.Linear(input_size + hidden_size, output_size)
7         self.softmax = nn.LogSoftmax(dim=1)
8
9     def forward(self, input, hidden):
10        combined = torch.cat((input, hidden), 1)
11        hidden = self.i2h(combined)
12        output = self.i2o(combined)
13        output = self.softmax(output)
14        return output, hidden
15
```

网络结构输出如下。

```
RNN(
  (i2h): Linear(in_features=185, out_features=128, bias=True)
  (i2o): Linear(in_features=185, out_features=18, bias=True)
  (softmax): LogSoftmax(dim=1)
)
```

3.2 训练流程

训练时对姓名中的字符逐个输入网络，最后一个字符对应的输出作为整个人名的分类结果。损失函数采用 NLLLoss，参数更新采用原始教程中的手动 SGD 方式。

```
1 for iter in range(1, n_iters + 1):
2     category, line, category_tensor, line_tensor = randomTrainingExample()
3     output, loss = train(category_tensor, line_tensor)
```

```
4
5 if iter % plot_every == 0:
6     val_loss, val_acc = evaluate_examples(val_examples)
7     all_losses.append(current_loss / plot_every)
8     val_losses.append(val_loss)
9     val accuracies.append(val_acc)
10
```

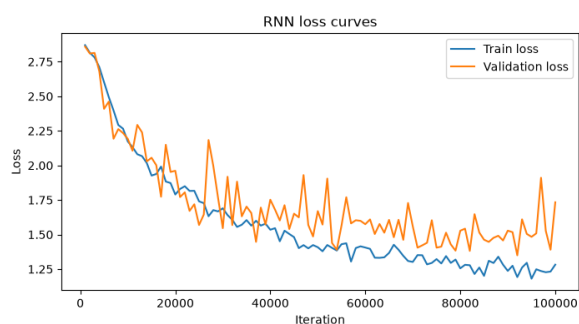
3.3 结果与分析

RNN 训练 100000 次迭代后的关键输出如下。

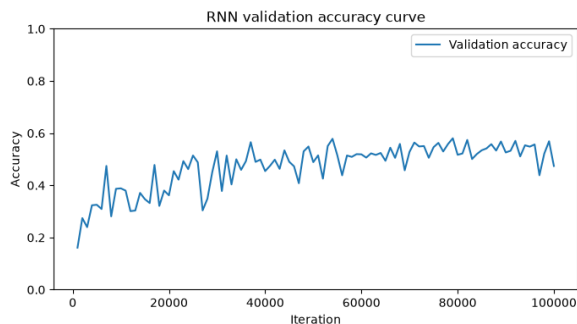
```
75000 75% (14m 24s) train_loss=0.0252 val_loss=1.4052 val_acc=54.63% Robertson / Scottish correct
95000 95% (16m 41s) train_loss=0.1395 val_loss=1.4833 val_acc=54.83% Nervetti / Italian correct
100000 100% (17m 15s) train_loss=0.8373 val_loss=1.7335 val_acc=47.42% Giroux / French correct
```

Saved RNN results:

```
final validation loss = 1.7335
final validation accuracy = 47.42%
```



(a) RNN loss 曲线



(b) RNN 验证准确率曲线

图 3.1: 原始 RNN 训练曲线

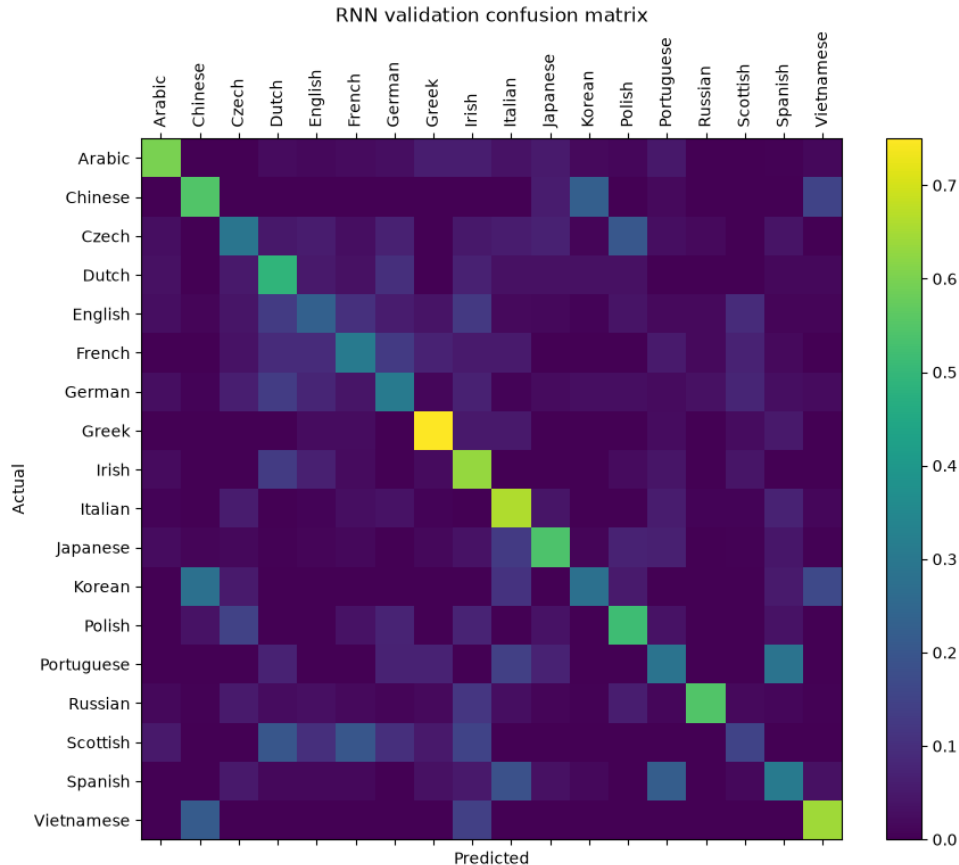


图 3.2: 原始 RNN 在验证集上的预测矩阵

从曲线看, RNN 在前期可以较快学习到姓名后缀和字符组合中的语种特征, 验证准确率最高达到约 54.83%。但后期验证 loss 有反弹, 最终准确率回落到 47.42%, 说明普通 RNN 在长序列依赖建模和稳定训练方面存在不足。混淆矩阵中可以看到若干语种之间存在明显混淆, 例如英文、苏格兰语、爱尔兰语、法语等姓名拼写模式较接近, 模型容易将这些类别互相误判。

4 LSTM

4.1 网络结构

LSTM 部分为个人手写实现, 没有调用 `nn.LSTM`。模型将输入 x_t 和上一隐藏状态 h_{t-1} 拼接, 一次线性变换得到 4 组门控向量, 分别对应遗忘门、输入门、候选记忆和输出门。

```

1 class NameLSTM(nn.Module):
2     def __init__(self, input_size, hidden_size, output_size):
3         super(NameLSTM, self).__init__()
4         self.hidden_size = hidden_size
5         self.i2gates = nn.Linear(input_size + hidden_size, hidden_size * 4)
6         self.h2o = nn.Linear(hidden_size, output_size)
7         self.softmax = nn.LogSoftmax(dim=1)
8
9     def forward(self, input, hidden, cell):
10        combined = torch.cat((input, hidden), 1)

```

```

11     gates = self.i2gates(combined)
12     forget_gate, input_gate, candidate_gate, output_gate = gates.chunk(4, dim=1)
13
14     forget_gate = torch.sigmoid(forget_gate)
15     input_gate = torch.sigmoid(input_gate)
16     candidate_gate = torch.tanh(candidate_gate)
17     output_gate = torch.sigmoid(output_gate)
18
19     cell = forget_gate * cell + input_gate * candidate_gate
20     hidden = output_gate * torch.tanh(cell)
21     output = self.softmax(self.h2o(hidden))
22     return output, hidden, cell
23

```

网络结构输出如下。

```

NameLSTM(
  (i2gates): Linear(in_features=185, out_features=512, bias=True)
  (h2o): Linear(in_features=128, out_features=18, bias=True)
  (softmax): LogSoftmax(dim=1)
)

```

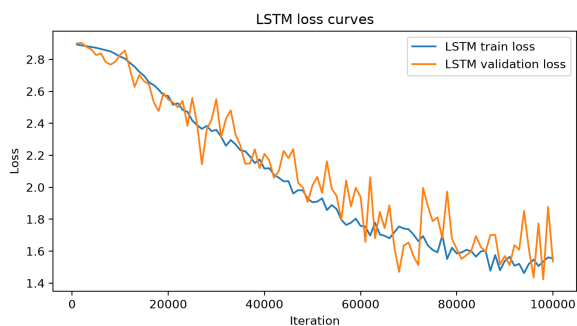
4.2 训练结果

LSTM 使用与 RNN 完全相同的训练集和验证集。训练 100000 次迭代后的关键输出如下。

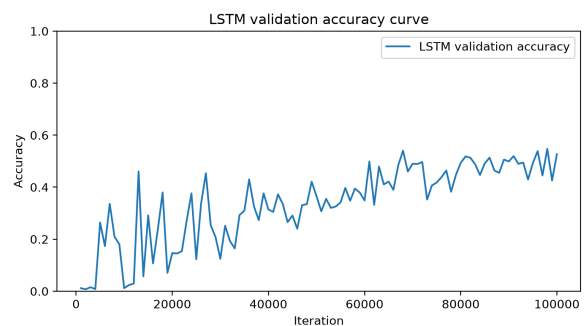
```

70000 70% (13m 46s) train_loss=5.4620 val_loss=1.6532 val_acc=48.99% Kwang / Japanese wrong
(Korean)
90000 90% (17m 41s) train_loss=2.6638 val_loss=1.5696 val_acc=49.89% Keay / Arabic wrong (English)
100000 100% (19m 38s) train_loss=1.5251 val_loss=1.5360 val_acc=52.66% Kirwin / Scottish wrong
(English)

```



(a) LSTM loss 曲线



(b) LSTM 验证准确率曲线

图 4.3: 个人实现 LSTM 训练曲线

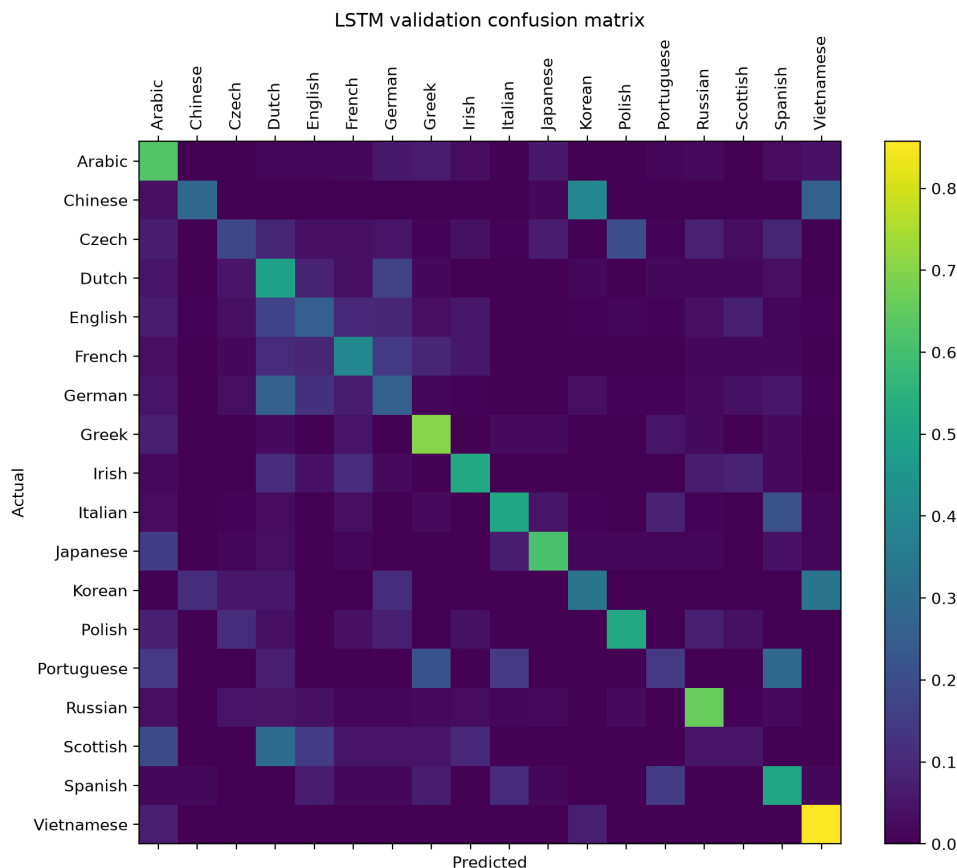


图 4.4: 个人实现 LSTM 在验证集上的预测矩阵

LSTM 最终验证 loss 为 1.5360, 验证准确率为 52.66%, 相较 RNN 最终结果有明显提升。曲线显示 LSTM 在中后期仍能继续降低验证 loss, 准确率整体上升更稳定。预测矩阵中主对角线较 RNN 更明显, 说明在多个语种上分类能力有所增强。

5 LSTM 优势分析

普通 RNN 的隐藏状态更新可以写成

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b),$$

它将所有历史信息压缩在一个隐藏状态中。反向传播时梯度需要不断经过同一个递归变换, 容易出现梯度消失或梯度爆炸。对于姓名识别任务, 虽然单个姓名不算特别长, 但语种特征往往分布在不同位置, 例如前缀、后缀和特定字符组合都可能影响最终类别。普通 RNN 较难长期、稳定地保留这些信息。

LSTM 通过细胞状态 c_t 和门控机制缓解上述问题:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t).$$

其中遗忘门 f_t 控制旧信息保留程度, 输入门 i_t 控制新信息写入程度, 输出门 o_t 控制当前记忆向分类输出暴露的程度。这样模型不必在每个时间步完全重写隐藏状态, 而是可以在较长范围内保存有用字

符模式。

结合实验结果，RNN 最终验证准确率为 47.42%，LSTM 最终验证准确率为 52.66%；RNN 最终验证 loss 为 1.7335，LSTM 为 1.5360。LSTM 的提升说明门控记忆结构能够更好地利用姓名字符序列中的上下文信息，减少后期训练震荡，使分类边界更稳定。

6 GRU

6.1 网络结构

GRU 同样为个人手写实现，没有调用 `nn.GRU`。与 LSTM 相比，GRU 没有单独的细胞状态，而是使用更新门和重置门控制隐藏状态的更新。

```

1 class NameGRU(nn.Module):
2     def __init__(self, input_size, hidden_size, output_size):
3         super(NameGRU, self).__init__()
4         self.hidden_size = hidden_size
5         self.i2z = nn.Linear(input_size + hidden_size, hidden_size)
6         self.i2r = nn.Linear(input_size + hidden_size, hidden_size)
7         self.i2n = nn.Linear(input_size + hidden_size, hidden_size)
8         self.h2o = nn.Linear(hidden_size, output_size)
9         self.softmax = nn.LogSoftmax(dim=1)
10
11     def forward(self, input, hidden):
12         combined = torch.cat((input, hidden), 1)
13         update_gate = torch.sigmoid(self.i2z(combined))
14         reset_gate = torch.sigmoid(self.i2r(combined))
15
16         reset_combined = torch.cat((input, reset_gate * hidden), 1)
17         candidate_hidden = torch.tanh(self.i2n(reset_combined))
18         hidden = update_gate * hidden + (1 - update_gate) * candidate_hidden
19
20         output = self.softmax(self.h2o(hidden))
21         return output, hidden
22

```

网络结构输出如下。

```

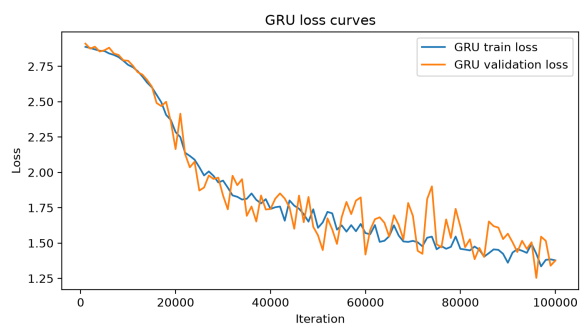
NameGRU(
  (i2z): Linear(in_features=185, out_features=128, bias=True)
  (i2r): Linear(in_features=185, out_features=128, bias=True)
  (i2n): Linear(in_features=185, out_features=128, bias=True)
  (h2o): Linear(in_features=128, out_features=18, bias=True)
  (softmax): LogSoftmax(dim=1)
)

```

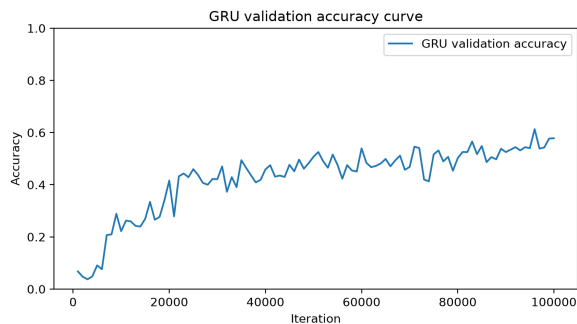
6.2 训练结果

GRU 使用与 RNN、LSTM 相同的训练集和验证集。训练 100000 次迭代后的关键输出如下。

```
60000 60% (14m 41s) train_loss=1.2292 val_loss=1.4188 val_acc=53.93% Matos / Greek wrong
(Portuguese)
85000 85% (20m 48s) train_loss=0.9392 val_loss=1.4016 val_acc=54.83% Dufour / Arabic wrong
(French)
100000 100% (24m 30s) train_loss=0.0011 val_loss=1.3761 val_acc=57.83% Panayiotopoulos / Greek
correct
```



(a) GRU loss 曲线



(b) GRU 验证准确率曲线

图 6.5: 个人实现 GRU 训练曲线

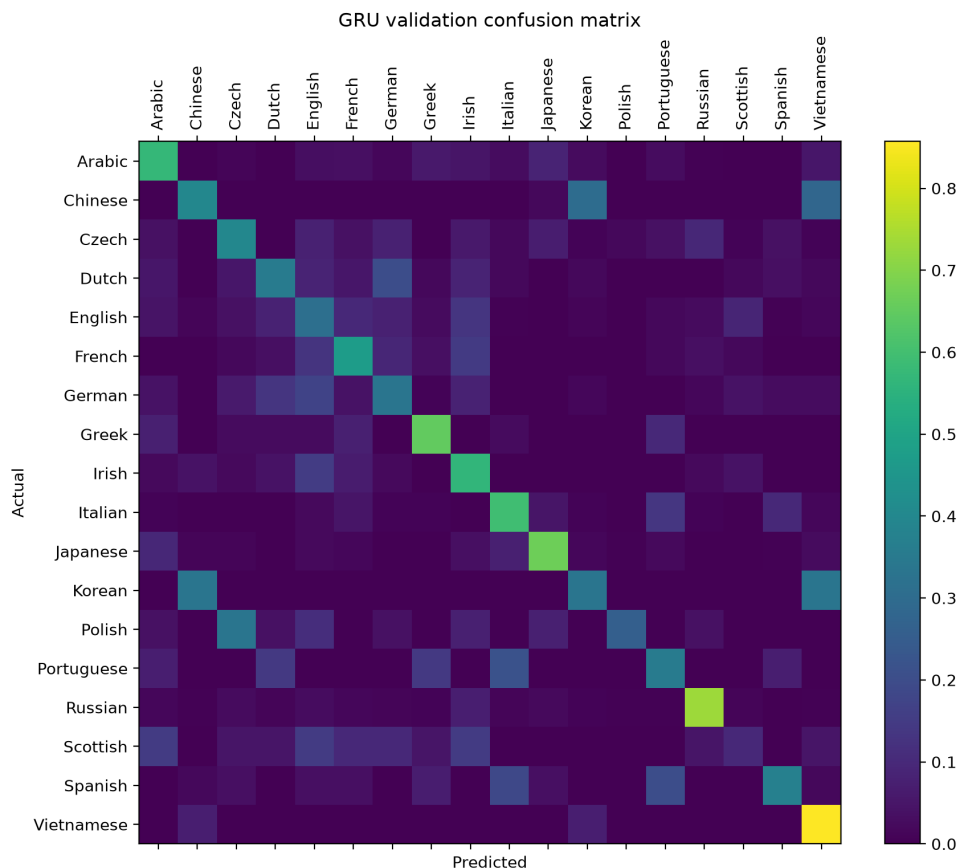


图 6.6: 个人实现 GRU 在验证集上的预测矩阵

GRU 最终验证 loss 为 1.3761, 验证准确率为 57.83%, 是三种模型中最好的结果。其训练时间略长于 RNN 和 LSTM, 但在本实验中泛化效果最好。GRU 结构比 LSTM 更简洁, 没有额外的细胞状态, 在该姓名分类任务上能够以较少门控参数获得更好的验证表现。

7 对比分析

7.1 结构对比

三种模型的核心差异如表1所示。

表 1: RNN、LSTM 和 GRU 结构对比

模型	主要状态	门控机制	输出层
RNN	隐藏状态 h_t	无	Linear + LogSoftmax
LSTM	隐藏状态 h_t 和细胞状态 c_t	遗忘门、输入门、输出门	Linear + LogSoftmax
GRU	隐藏状态 h_t	更新门、重置门	Linear + LogSoftmax

训练伪代码如下, 三种模型的主要区别在于 `run_model` 内部的状态更新公式不同。

```

1 for iteration in range(1, 100000 + 1):
2     category, line = sample_from_training_set()
3     line_tensor = lineToTensor(line)

```

```

4     target = categoryToTensor(category)
5
6     output = run_model(line_tensor)
7     loss = NLLLoss(output, target)
8     loss.backward()
9     update_parameters_with_sgd()
10
11     if iteration % 1000 == 0:
12         val_loss, val_acc = evaluate_on_same_validation_set()
13

```

7.2 指标对比

三种模型最终验证集结果如下。

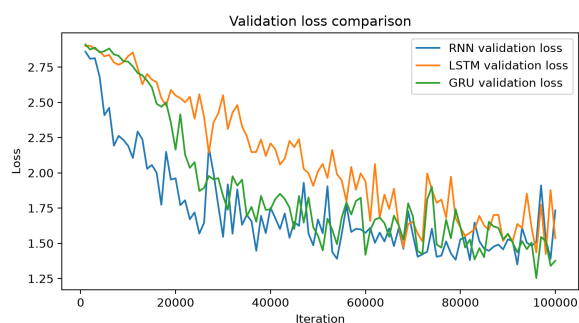
```

Final RNN validation accuracy: 47.42%
Final LSTM validation accuracy: 52.66%
Final GRU validation accuracy: 57.83%
Final RNN validation loss: 1.7335
Final LSTM validation loss: 1.5360
Final GRU validation loss: 1.3761

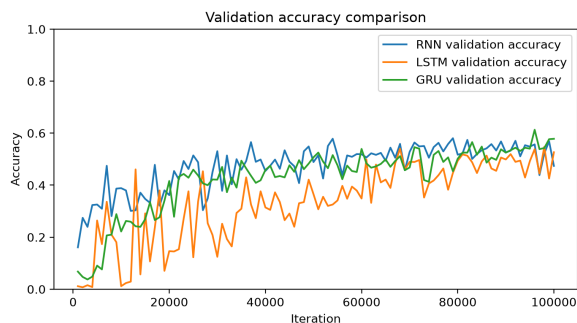
```

表 2: 最终验证集结果对比

模型	验证 loss	验证准确率
RNN	1.7335	47.42%
LSTM	1.5360	52.66%
GRU	1.3761	57.83%



(a) 三模型验证 loss 对比



(b) 三模型验证准确率对比

图 7.7: RNN、LSTM 和 GRU 曲线对比

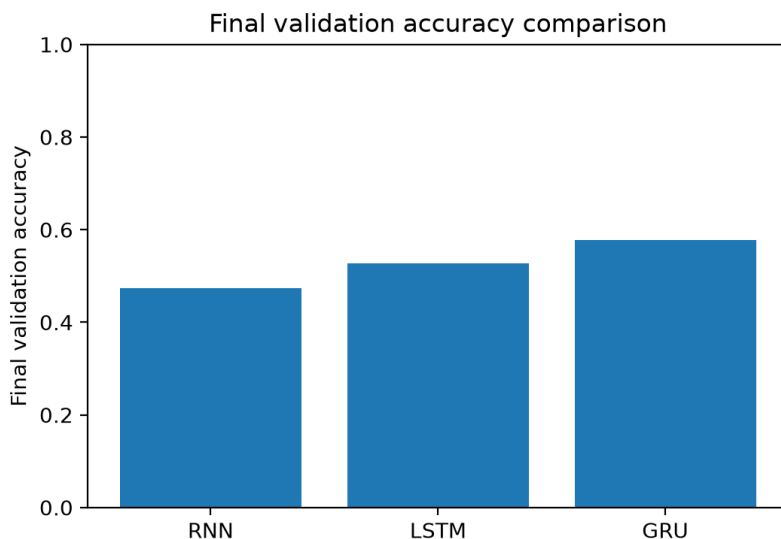


图 7.8: RNN、LSTM 和 GRU 最终验证准确率对比

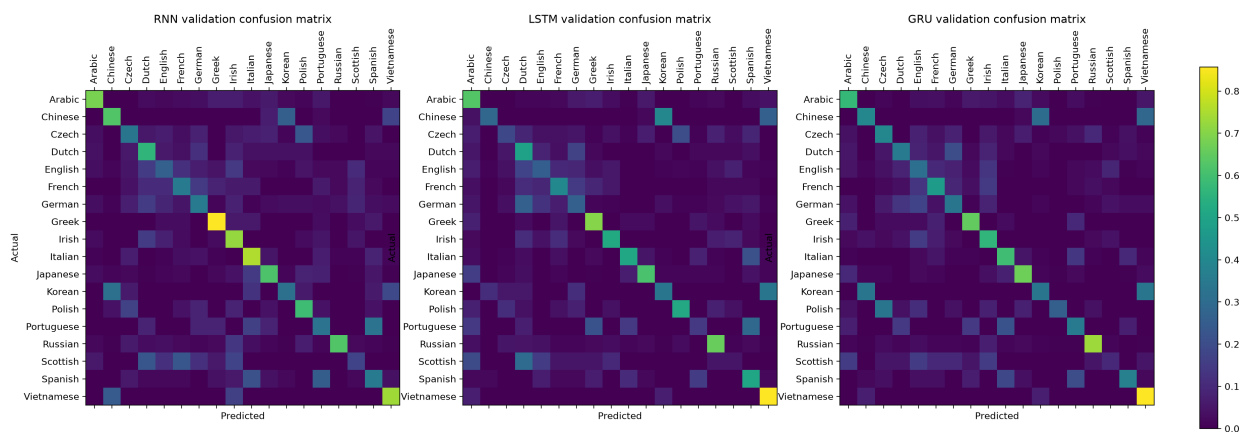


图 7.9: RNN、LSTM 和 GRU 验证集预测矩阵对比

从结果看，GRU > LSTM > RNN。RNN 没有门控结构，容易受当前字符和短期模式影响，验证集曲线震荡较大。LSTM 引入细胞状态和三个门，能保留更长期的字符模式，因此优于 RNN。GRU 用更新门和重置门实现对隐藏状态的选择性保留与重置，结构比 LSTM 更简单，在本实验中训练后期验证 loss 继续下降，最终准确率最高。

7.3 预测样例

训练结束后对三个姓名进行预测，输出如下。

```
> Dovesky
(0.57) Czech
(0.23) Russian
(0.06) English

> Jackson
(0.29) Scottish
```

```
(0.27) English
(0.14) French

> Hou
(0.54) Korean
(0.28) Chinese
(0.05) Vietnamese
```

可以看到，模型对带有明显地区拼写特征的姓名会给出较高置信度。例如 **Hou** 被预测为 **Korean** 和 **Chinese** 的概率最高，符合东亚姓名拼写特征；**Jackson** 在 **Scottish** 和 **English** 之间概率接近，也反映了这些类别在姓名拼写上的相似性。

8 实验心得

本次实验从原始 RNN 出发，逐步补充验证集划分、指标记录、混淆矩阵评估以及 LSTM 和 GRU 手写实现。实验过程中我体会到，循环神经网络并不只是把序列逐项输入模型，更关键的是如何设计状态更新方式。普通 RNN 结构简单，但隐藏状态缺乏选择性记忆机制，在训练后期容易震荡；LSTM 和 GRU 通过门控机制控制信息保留和遗忘，能够更稳定地利用姓名中的前缀、后缀和字符组合信息。

此外，本次实验中我单独记录训练 loss、验证 loss、验证准确率和混淆矩阵，可以从多个角度判断模型是否真正泛化，而不是只记住训练样本，这让我对评估和分析模型的操作有所理解。最终实验结果表明，在本任务中 GRU 取得了最低验证 loss 和最高验证准确率，说明更简洁的门控结构在小规模字符级分类任务上也能取得较好的效果。